

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2024-2025

Class: BE-ITA/B, Semester: VIII

Subject: BlockChain Lab

Experiment –2

1. **Aim:** To implement smart contract using Remix IDE

2. **Objective:** To ...

- explain what smart contracts are.
- understand coding smart contracts using solidity language.
- test a smart contract by using Remix IDE

3. **Lab outcome:** After performing the experiment, the students will be able to **implement** smart contracts in Ethereum using different development frameworks (PO3, PSO2, BL3)

4. **Prerequisite:**

- Fundamental knowledge of blockchain
- Knowledge of the Ethereum platform and Remix IDE
- Familiarity with the Solidity programming language

5. **Requirements:** The following are the requirements –

- Remix IDE

6. **Pre-Experiment Theory:**

What is Blockchain?

Blockchain is a shared, immutable ledger that facilitates the process of recording transactions and tracking assets in a business network. An asset can be tangible (a house, car, cash, land) or intangible (intellectual property, patents, copyrights, branding). Virtually anything of value can be tracked and traded on a blockchain network, reducing risk and cutting costs for all involved.

What is Ethereum?

Ethereum is a decentralized blockchain designed to be highly secure, fault-tolerant, and programmable. Ethereum blockchain is a choice for many developers and businesses. As said programmable, the main task of Ethereum is to securely execute and verify the application code known as **smart contracts**. Ethereum helps to build native scripting languages (solidity) and EVM. Ethereum consensus mechanism is proof of work to operate to verify the new transaction.

What is Remix IDE?

It is an online IDE for creating solid, **smart contracts**, so you do not need to install or download anything to do any setup. You can develop, deploy, and administer your solidity smart contract using Remix IDE.

What is Solidity?

Solidity is a statically typed, contract-oriented, high-level language for implementing smart contracts that run on the Ethereum Virtual Machine. Smart contracts are programs that are executed inside a peer-to-peer network where nobody has special authority over the execution, and thus they allow anyone to implement tokens of value, ownership, voting, and other kinds of logic.

What is a smart contract?

A smart contract is a small program that runs on an Ethereum blockchain. Once the smart contract is deployed on the Ethereum blockchain, it cannot be changed. To deploy the smart contract to Ethereum, you must pay the ether (ETH) cost. Understand it as a digital agreement that builds trust and allows both parties to agree on a particular set of conditions that cannot be tampered with.

7. Laboratory Exercise

A. Steps to be implemented.

To Follow the procedure given below to build smart contract in Remix IDE

1. Open Remix IDE in Google Chrome
2. Write a smart contract by creating a new file under contract folder, with .sol extension.
3. Write your contract code using solidity language.
4. Click on the Solidity compiler icon.
5. Choose the compiler version or keep default and click on the compile button.
6. Click on Deploy and Run Transaction icon.
7. Choose Environment Remix VM (London). and choose Account Number or keep default.
8. Click on deploy button to deploy smart contract on Ethereum blockchain.
9. Under the deployed contract, get the output of your contract.

B. Program Code

1. Write the first smart contract HelloWorld.sol as follows.

```
=====
// SPDX - License - identifier:
GPL-3.0 pragma solidity >=0.7.0
<0.9.0; contract HelloWorld {
function Greet() public view returns(string memory){
    return "Hello Everyone, I am Joanne";
}
}
=====
```

2. Write a smart contract storage.sol to store a number and to retrieve the number as follows.

```
=====
// SPDX-License-Identifier:
GPL-3.0 pragma solidity >=0.7.0
<0.9.0; contract storage1 {
uint number;
function storeInt (uint _num) public {
    number = _num; }
function retrieve () public view returns (uint) {
    return number;
}
}
=====
```

3. Write a smart contract employee.sol to create an employee database with fields like empId, empName, empDept, empDesignation. Create functions to addEmployee and getEmployee. Use concept of structure.

8. Post Experimental Exercise-

C. Questions:

1. Write down key properties and advantages of smart contracts
2. Create a Hostel.sol smart contract by following the tutorial from

<https://blog.loginradius.com/engineering/guest-post/ethereum-smart-contract-tutorial/>

D. Results/Observations/Program output:

Present the program input/output results if any and comment on the same.

E. Conclusion:

1. Write what was performed in the experiment
2. Write which tools you used to perform the experiment
3. Write what you inferred from the output obtained

F. References:

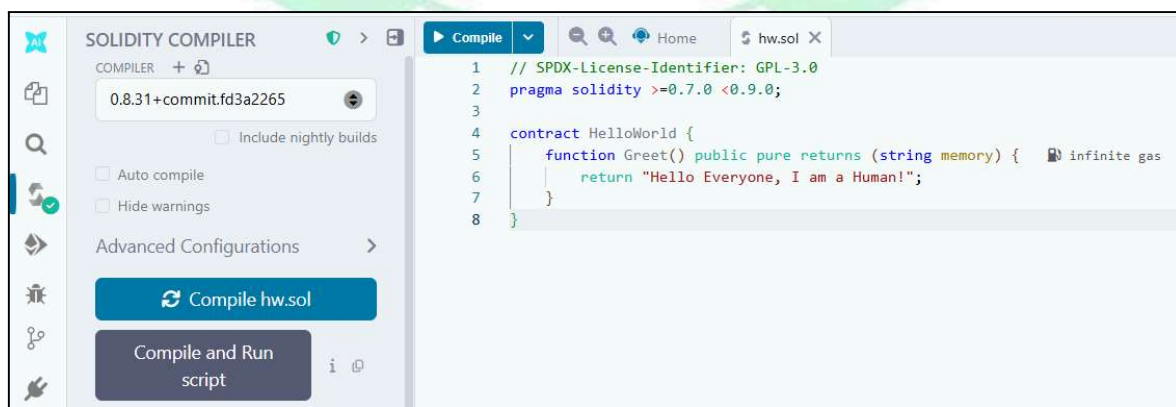
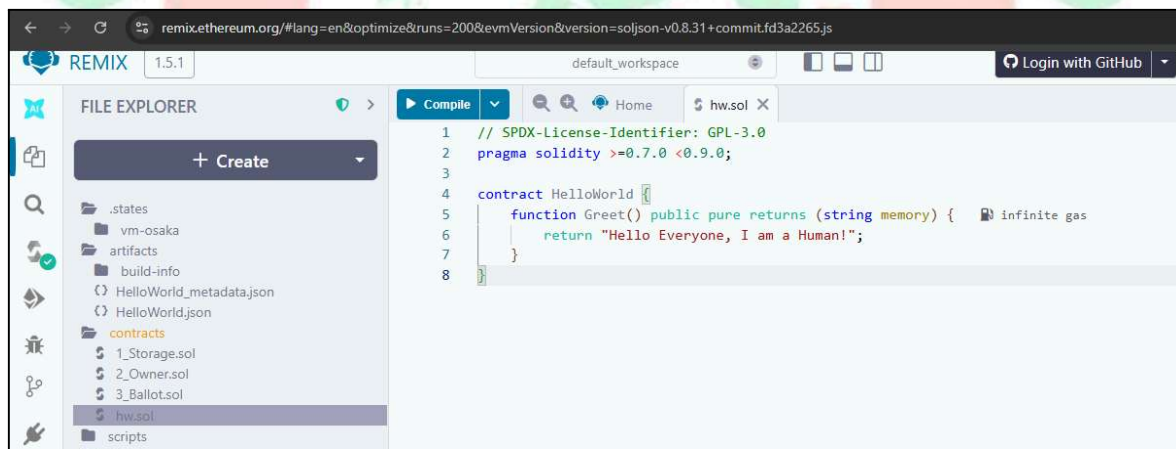
- [1] <https://www.simplilearn.com/tutorials/blockchain-tutorial/what-is-smart-contract>
- [2] <https://ethereum.org/en>
- [3] Mastering Ethereum, Building Smart Contract and Dapps, Andreas M. Antonopoulos Dr. Gavin Wood, O'reilly

Output:

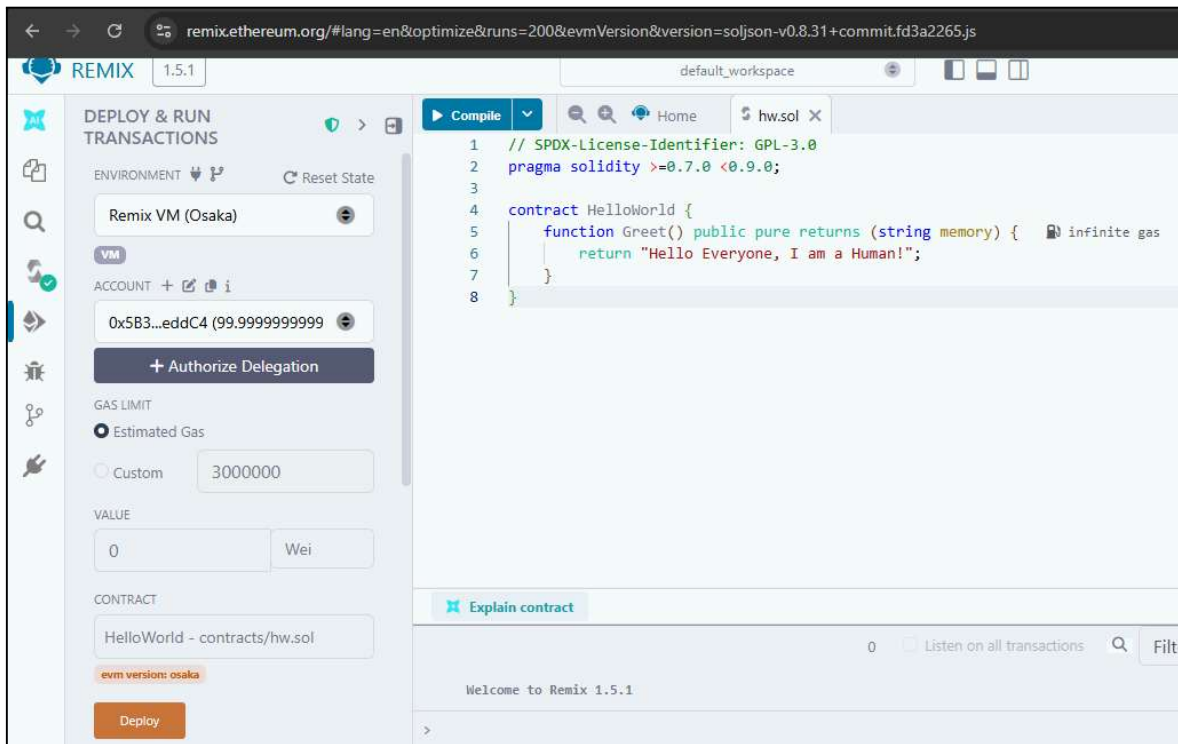
#1 code

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

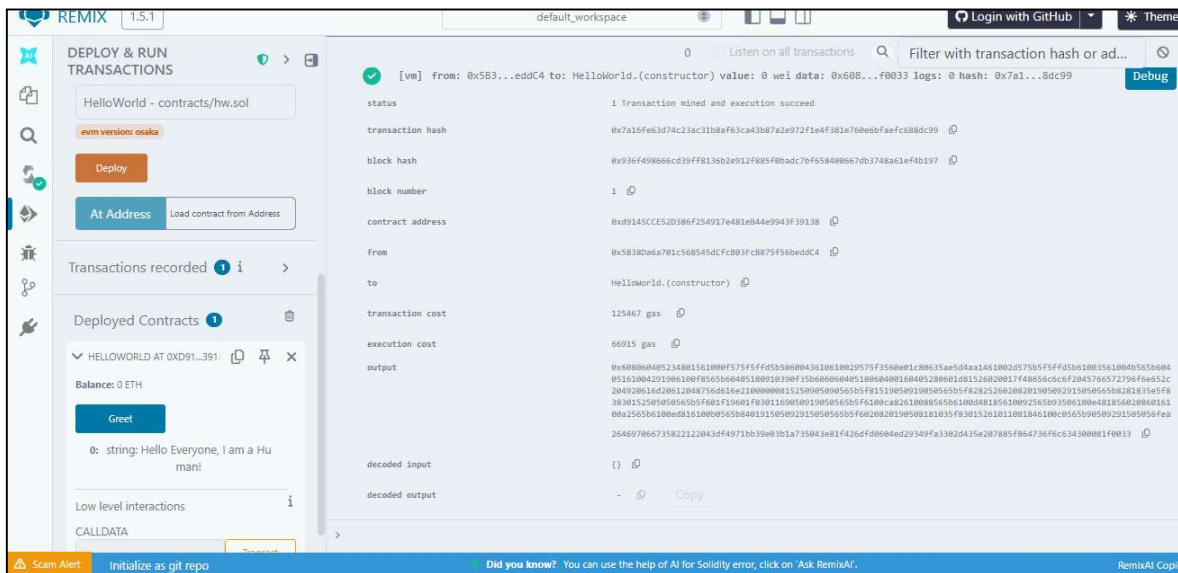
contract HelloWorld {
    function Greet() public pure returns (string memory) {
        return "Hello Everyone, I am a Human!";
    }
}
```



Compiling the code



Deploying the code, keeping settings as default



Output after deploy (hashed value)



Output after calling the Greet() function, we get output

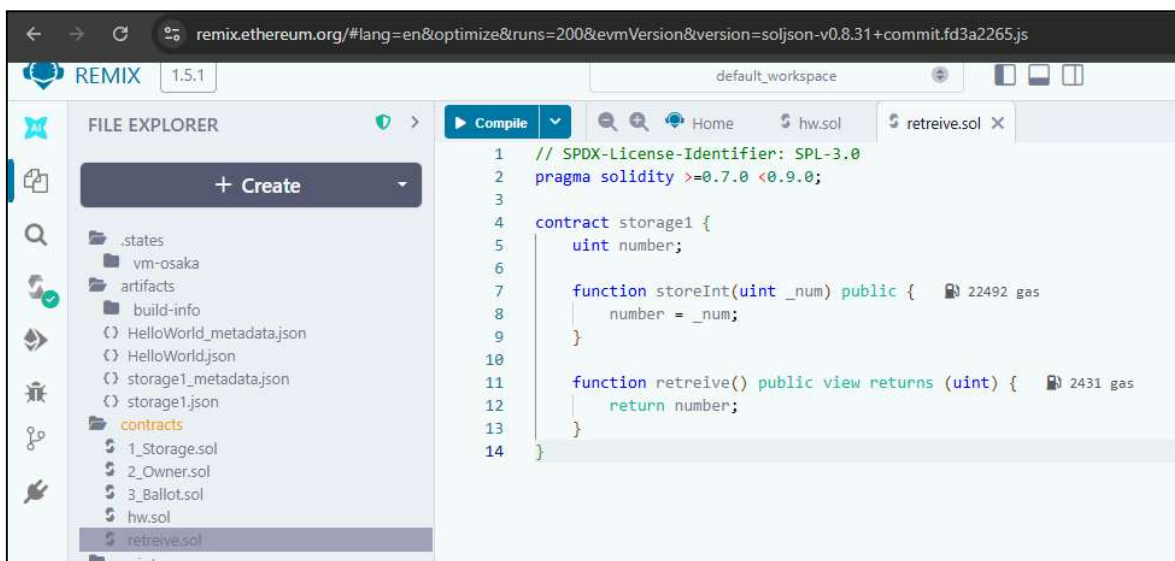
#2 code

```
// SPDX-License-Identifier: SPL-3.0
pragma solidity >=0.7.0 <0.9.0;
```

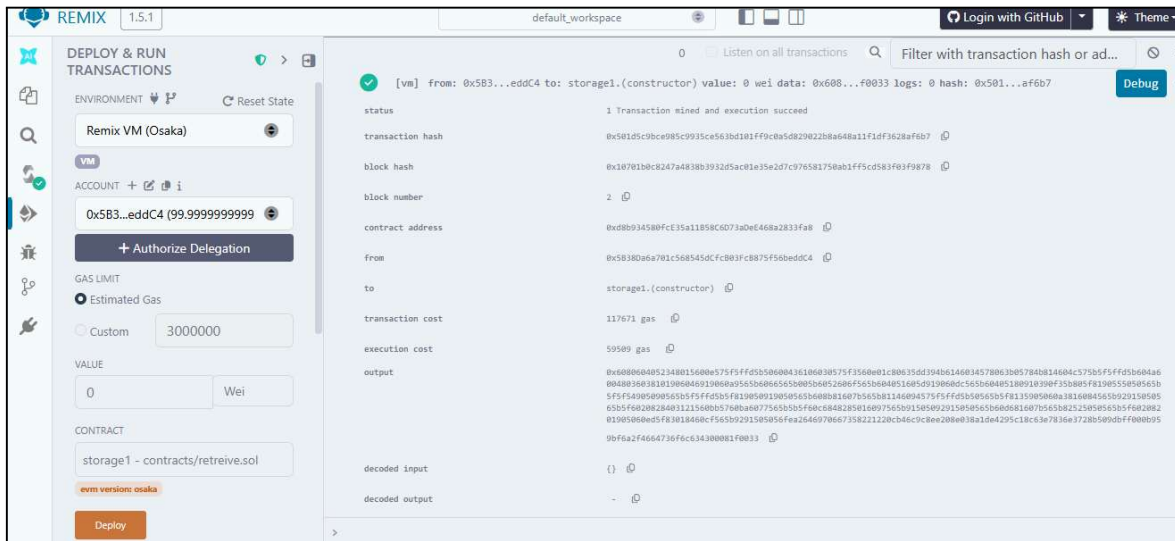
```
contract storage1 {
    uint number;

    function storeInt(uint _num) public {
        number = _num;
    }

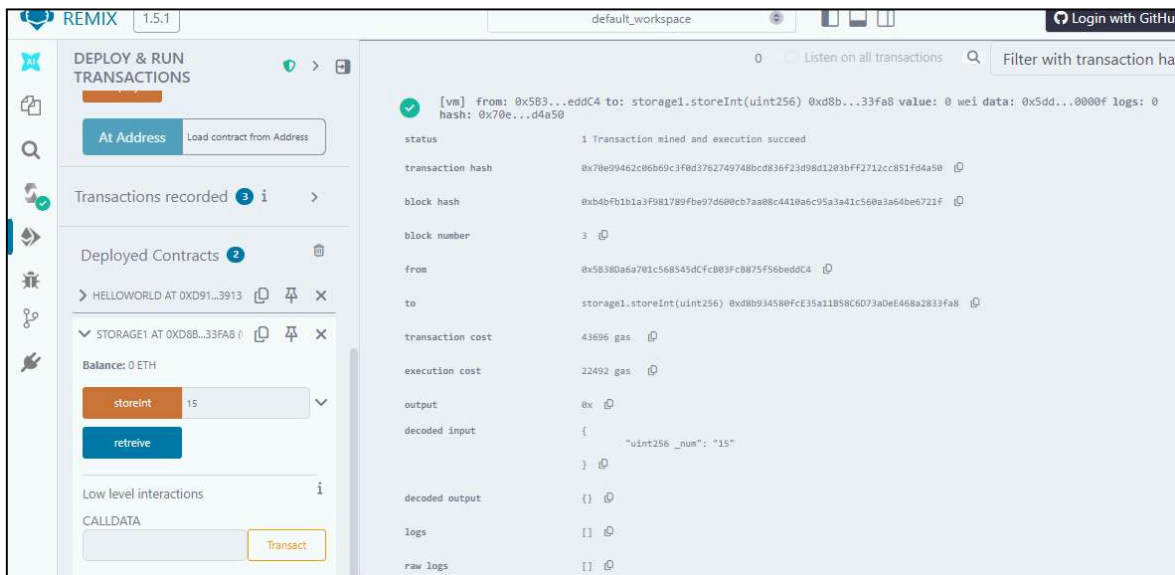
    function retrieve() public view returns (uint) {
        return number;
    }
}
```



Writing the code then compiling and deploying



Deployed successfully



In deployed contracts, setting storeInt as 15



Using retrieve() function and getting the output

#3 code

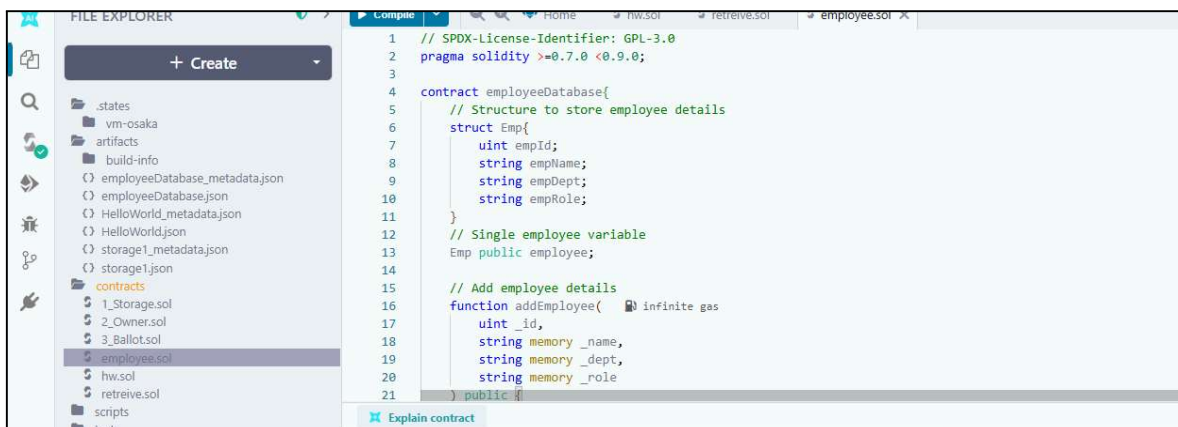
Write a smart contract `employee.sol` to create an employee database with fields like `empId`, `empName`, `empDept`, `empDesignation`. Create functions to `addEmployee` and `getEmployee`. Use concept of structure

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract employeeDatabase{
    // Structure to store employee details
    struct Emp{
        uint empId;
        string empName;
        string empDept;
        string empRole;
    }
    // Single employee variable
    Emp public employee;

    // Add employee details
    function addEmployee(
        uint _id,
        string memory _name,
        string memory _dept,
        string memory _role
    ) public {
        employee = Emp(_id, _name, _dept, _role);
    }

    // Get employee details
    function getEmployee() public view returns (uint, string memory, string
memory, string memory){
        return(
            employee.empId,
            employee.empName,
            employee.empDept,
            employee.empRole
        );
    }
}
```



Writing the code and compiling

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT: Remix VM (Osaka)

ACCOUNT: 0x5B3...eddC4 (99.999999999)

GAS LIMIT: Estimated Gas (3000000)

VALUE: 0 Wei

CONTRACT: employeeDatabase - contracts/empli

Transaction status: 1 Transaction mined and execution succeed

Transaction hash: 0xed179daadcb49128dd80e04e93c33a24d327b8f8d82332d5a8241f97d86bcc59b

Block hash: 0x7c2ae97487935d38f5ba464240ac7c7f5ab3c85cb917736ed246651dc30c8d9

Block number: 4

Contract address: 0xd7ACd2a9FD159E698b102A1ca21C9a3e3A5F771B

From: 0x5B38D6a781c568545dcfcb83fc8875f56beddC4

To: employeeDatabase.(constructor)

Transaction cost: 63533 gas

Execution cost: 53953 gas

Output: 0x08006005234001561000f57575ff5d508004361861001f57575f3560e01c80634132db8f14610043578063a0a1a7d714610064578063de9... (long hexadecimal string)

Decoded input: {}

Decoded output: {}

Logs: []

Raw logs: []

Deploying the code

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT: Remix VM (Osaka)

ACCOUNT: 0x5B3...eddC4 (99.999999999)

GAS LIMIT: Estimated Gas (3000000)

VALUE: 0 Wei

CONTRACT: employeeDatabase - contracts/empli

Transaction status: 1 Transaction mined and execution succeed

Transaction hash: 0x5dc577864c72d411a9a5e736f0898d744d6cac5628621861db095119a13515a

Block hash: 0x5c75dfe4d2539adc4d3f790ab86d003f2a4980c998c93d1c84b22bc7e40cf

Block number: 5

Contract address: 0xd7ACd2a9FD159E698b102A1ca21C9a3e3A5F771B

From: 0x5B38D6a781c568545dcfcb83fc8875f56beddC4

To: employeeDatabase.addEmployee(uint256,string,string,string)

Transaction cost: 115667 gas

Execution cost: 92915 gas

Output: 0x

Decoded input: { "uint256_id": "1", "string_name": "styris", "string_dept": "INFT", "string_role": "Software Engineer" }

Decoded output: {}

Adding the employee record

The screenshot displays a transaction interface with a left sidebar and a main content area. The sidebar, titled 'DEPLOY & RUN TRANSACTIONS', shows a balance of 0 ETH and a dropdown menu set to '1. styris, INFT, Software Eng'. It contains buttons for 'addEmployee' and 'employee'. Below these, a list of parameters is shown: '0: uint256: empId 1', '1: string: empName styris', '2: string: empDept INFT', and '3: string: empRole Software Engineer'. A 'getEmployee' button is also present. The main content area shows the transaction details for a call from 0x58380a6a701c568545dcfc803fc8b75f56beddC4 to employeeDatabase.employee(). The execution cost is 11752 gas. The decoded output is a JSON object: {"0": "uint256: empId 1", "1": "string: empName styris", "2": "string: empDept INFT", "3": "string: empRole Software Engineer"}. A 'Debug' button is in the top right corner.

Employee information

The screenshot displays a transaction interface similar to the one above. The sidebar shows the same balance and dropdown menu. The 'getEmployee' button is highlighted. The main content area shows the transaction details for a call from 0x58380a6a701c568545dcfc803fc8b75f56beddC4 to employeeDatabase.getEmployee(). The execution cost is 11774 gas. The decoded output is a JSON object: {"0": "uint256: 1", "1": "string: styris", "2": "string: INFT", "3": "string: Software Engineer"}. A 'Debug' button is in the top right corner.

Calling the getEmployee() function and getting successful output

Post Experiment code

```
// SPDX-License-Identifier: GPL-3.0
```

```
pragma solidity >=0.7.0 <0.9.0;
```

```
contract Hostel {
```

```
    uint256 public no_of_rooms = 0;
    uint256 public no_of_agreement = 0;
    uint256 public no_of_rent = 0;
    bool private locked;
```

```
    mapping(uint256 => uint256) public depositsHeld;
```

```
struct Room {
    uint roomid;
    uint agreementid;
    string roomname;
    string roomaddress;
    uint rent_per_month;
    uint securityDeposit;
    uint timestamp;
    bool vacant;
    address payable landlord;
    address payable currentTenant;
}
mapping(uint => Room) public Room_by_No;

struct RoomAgreement {
    uint roomid;
    uint agreementid;
    string Roomname;
    string RoomAddresss;
    uint rent_per_month;
    uint securityDeposit;
    uint lockInPeriod;
    uint timestamp;
    address payable tenantAddress;
    address payable landlordAddress;
}
mapping(uint => RoomAgreement) public RoomAgreement_by_No;

struct Rent {
    uint rentno;
    uint roomid;
    uint agreementid;
    string Roomname;
    string RoomAddresss;
    uint rent_per_month;
    uint timestamp;
    address payable tenantAddress;
    address payable landlordAddress;
}
mapping(uint => Rent) public Rent_by_No;

modifier noReentrancy() {
    require(!locked, "Reentrancy detected");
    locked = true;
    _;
    locked = false;
}

modifier onlyLandlord(uint _index) {
    require(msg.sender == Room_by_No[_index].landlord, "Not Landlord");
    _;
}

modifier notLandLord(uint _index) {
    require(msg.sender != Room_by_No[_index].landlord, "Landlord cannot be Tenant");
    _;
}
```

```

modifier OnlyWhileVacant(uint _index){
    require(Room_by_No[_index].vacant == true, "Room Occupied");
    _;
}

modifier validRoom(uint _index) {
    require(_index > 0 && _index <= no_of_rooms, "Room invalid");
    _;
}

modifier enoughAgreementfee(uint _index) {
    uint total = Room_by_No[_index].rent_per_month + Room_by_No[_index].securityDeposit;
    require(msg.value >= total, "Not enough Ether");
    _;
}

modifier sameTenant(uint _index) {
    require(msg.sender == Room_by_No[_index].currentTenant, "Not current tenant");
    _;
}

modifier AgreementTimesUp(uint _index) {
    uint _AgreementNo = Room_by_No[_index].agreementid;
    uint time = RoomAgreement_by_No[_AgreementNo].timestamp + RoomAgreement_by_No[_AgreementNo].lockInPeriod;
    require(block.timestamp > time, "Contract not yet ended");
    _;
}

function addRoom(string memory _roomname, string memory _roomaddress, uint _rentcost, uint _securitydeposit) public {
    require(msg.sender != address(0));
    no_of_rooms++;
    Room_by_No[no_of_rooms] = Room(no_of_rooms, 0, _roomname, _roomaddress, _rentcost, _securitydeposit, 0, true,
    payable(msg.sender), payable(address(0)));
}

function signAgreement(uint _index) public payable noReentrancy validRoom(_index) notLandLord(_index)
    enoughAgreementfee(_index) OnlyWhileVacant(_index) {
    require(msg.sender != address(0));

    uint _rent = Room_by_No[_index].rent_per_month;
    uint _deposit = Room_by_No[_index].securityDeposit;

    no_of_agreement++;
    depositsHeld[_index] = _deposit;

    Room_by_No[_index].currentTenant = payable(msg.sender);
    Room_by_No[_index].vacant = false;
    Room_by_No[_index].timestamp = block.timestamp;
    Room_by_No[_index].agreementid = no_of_agreement;

    RoomAgreement_by_No[no_of_agreement] = RoomAgreement(_index, no_of_agreement,
    Room_by_No[_index].roomname, Room_by_No[_index].roomaddress, _rent, _deposit, 365 days, block.timestamp,
    payable(msg.sender), Room_by_No[_index].landlord);

    no_of_rent++;
    Rent_by_No[no_of_rent] = Rent(no_of_rent, _index, no_of_agreement, Room_by_No[_index].roomname,
    Room_by_No[_index].roomaddress, _rent, block.timestamp, payable(msg.sender), Room_by_No[_index].landlord);

```

```

    (bool success, ) = Room_by_No[_index].landlord.call{value: _rent}("");
    require(success, "Rent transfer failed");
}

function payRent(uint _index) public payable noReentrancy validRoom(_index) sameTenant(_index) {
    require(msg.value >= Room_by_No[_index].rent_per_month, "Incorrect Rent");

    Room_by_No[_index].timestamp = block.timestamp;
    no_of_rent++;
    Rent_by_No[no_of_rent] = Rent(no_of_rent, _index, Room_by_No[_index].agreementid,
    Room_by_No[_index].roomname, Room_by_No[_index].roomaddress, msg.value, block.timestamp, payable(msg.sender),
    Room_by_No[_index].landlord);

    (bool success, ) = Room_by_No[_index].landlord.call{value: msg.value}("");
    require(success, "Rent transfer failed");
}

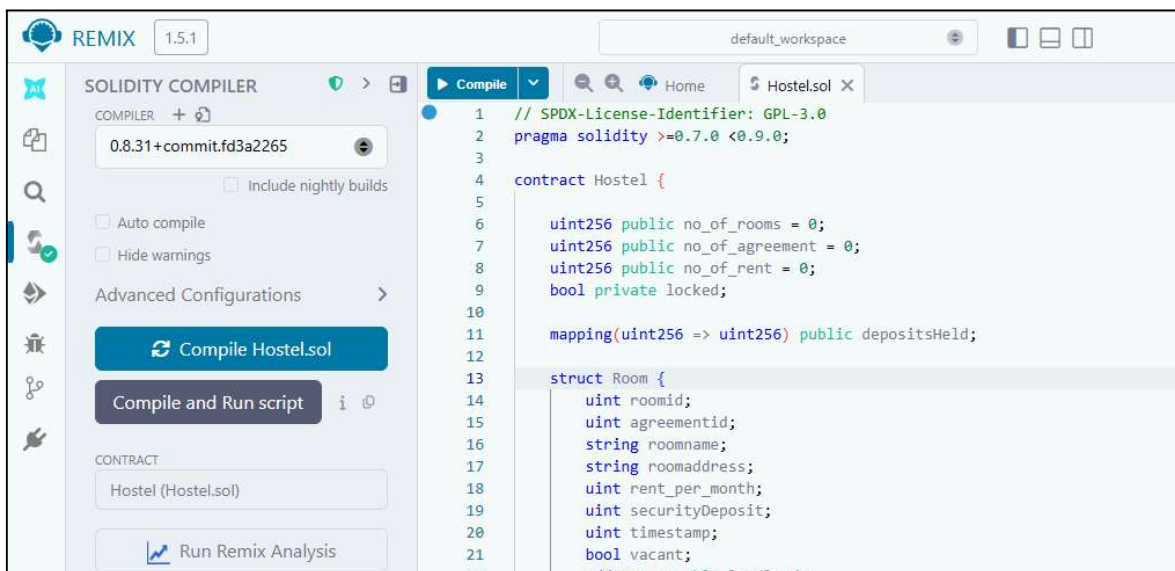
function agreementCompleted(uint _index) public noReentrancy validRoom(_index) onlyLandlord(_index)
AgreementTimesUp(_index) {
    require(Room_by_No[_index].vacant == false, "Room already vacant");

    uint refundAmount = depositsHeld[_index];
    require(refundAmount > 0, "No deposit to refund");

    depositsHeld[_index] = 0;
    Room_by_No[_index].vacant = true;
    address payable _Tenant = Room_by_No[_index].currentTenant;
    Room_by_No[_index].currentTenant = payable(address(0));

    require(address(this).balance >= refundAmount, "Insolvent Contract");
    (bool success, ) = _Tenant.call{value: refundAmount}("");
    require(success, "Refund failed");
}
}

```



Compiling the code

DEPLOY & RUN TRANSACTIONS

0 ☐ Listen on all transactions

transact to Hostel.signAgreement pending ...

✓ [vm] from: 0xAb8...35cb2 to: Hostel.signAgreement(uint256) 0x7EF...8CB47 value: 12 wei data: 0x hash: 0x82c...0e3f3

status 1 Transaction mined and execution succeed

transaction hash 0x82cfc63468f7f71df958407096e617658a4537cceb2ff7b5833dd8e4f40e3f3

block hash 0x58bb189fe11500962e1d4537f09a3952bc41f2e24a74ff538b9214cf5fa2d49

block number 10

from 0xAb8483f64d9C6d1EcF9b849Ae677d03315835cb2

to Hostel.signAgreement(uint256) 0x7EF2e0048f5bAeDe046f68f797943daf4ED8CB47

transaction cost 612362 gas

execution cost 611058 gas

output 0x

decoded input { "uint256 _index": "1" }

decoded output {}

logs []

raw logs []

value 12 wei

Left sidebar controls:

- Calldata Parameters **transact**
- agreementCo... uint256_index
- payRent uint256_index
- SIGNAGREEMENT**
- _index: 1
- Calldata Parameters **transact**
- depositsHeld uint256
- no_of_agreem...
- no_of_rent
- no_of_rooms
- Rent_by_No uint256
- Room_by_No uint256
- RoomAgreeme... uint256

Tenant successfully paid the rent + deposit.

DEPLOY & RUN TRANSACTIONS

0 ☐ Listen on all transactions

transact to Hostel.payRent pending ...

✓ [vm] from: 0xAb8...35cb2 to: Hostel.payRent(uint256) 0x7EF...8CB47 value: 2 wei data: 0xd

status 1 Transaction mined and execution succeed

transaction hash 0x97f5c08fa7bffa4a64484d714c70655714f3fcfeffa5900298a235e670482e599

block hash 0x06cfe7e3d919542018a747ceadd017440364feec76b2af43e3f3b619003d3d86

block number 13

from 0xAb8483f64d9C6d1EcF9b849Ae677d03315835cb2

to Hostel.payRent(uint256) 0x7EF2e0048f5bAeDe046f68f797943daf4ED8CB47

transaction cost 262123 gas

execution cost 260819 gas

output 0x

decoded input { "uint256 _index": "1" }

decoded output {}

logs []

raw logs []

value 2 wei

Left sidebar controls:

- HOSTEL AT 0x7EF...8CB47 (ME)
- Balance: 0.0000000000000001 ETH
- addRoom string_roomname, string_i
- agreementCo... uint256_index
- PAYRENT**
- _index: 1
- Calldata Parameters **transact**
- signAgreement uint256_index
- depositsHeld uint256
- no_of_agreem...
- no_of_rent
- no_of_rooms
- Rent_by_No uint256
- Room_by_No uint256
- RoomAgreeme... uint256

Tenant paying the rent 2 wei

DEPLOY & RUN TRANSACTIONS

HOSTEL AT 0X7EF...8CB47 (ME)   

Balance: 0.0000000000000001 ETH

addRoom string _roomname, string _roomaddress, uint256 _rentcost, uint256 _securitydeposit

agreementContract uint256 _index

payRent uint256 _index

signAgreement uint256 _index

depositsHeld uint256

no_of_agreement uint256

0: uint256: 1

no_of_rent uint256

0: uint256: 2

no_of_rooms uint256

0: uint256: 1

Rent_by_No uint256

Room_by_No uint256

RoomAgreement uint256

execution cost: 2469 gas (Cost only applies when called by a contract)

input: 0x75e...b9ef6

output: 0x0002

decoded input: {}

decoded output: {"0": "uint256: 2"}

logs: []

raw logs: []

call to Hostel.no_of_rooms

CALL [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 to: Hostel.no_of_rooms() data: 0x59e...2cbecc

from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2

to: Hostel.no_of_rooms() 0x7EF2e0048F5bAeDe046F6BF797943daF4ED8CB47

execution cost: 2424 gas (Cost only applies when called by a contract)

input: 0x59e...2cbecc

output: 0x0001

decoded input: {}

decoded output: {"0": "uint256: 1"}

Verifying the data for account created

DEPLOY & RUN TRANSACTIONS

Deployed Contracts 1   

HOSTEL AT 0X7EF...8CB47 (ME)   

Balance: 0.0000000000000001 ETH

ADDROOM

_roomname: "Suite B"

_roomaddress: "Delhi"

_rentcost: "3"

_securitydeposit: "10"

 Calldata  Parameters **transact**

agreementContract uint256 _index

payRent uint256 _index

signAgreement uint256 _index

depositsHeld uint256

0: uint256: 10

[vm] from: 0xAb8...35cb2 to: Hostel.addRoom(string,string,uint256,uint256) 0x7EF...8CB47 value: 0 wei data: 0x63e...9af4f

status: 1 Transaction mined and execution succeed

transaction hash: 0x2bf340d594ea521d4fee428b883cf686fe02c89d74d39c58cdc47c535879af4f

block hash: 0xe3d8efb1aa76f1756e740411d8427b2a07cha188Fbc975cb97d3da95e48e8391

block number: 14

from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2

to: Hostel.addRoom(string,string,uint256,uint256) 0x7EF2e0048F5bAeDe046F6BF797943daF4ED8CB47

transaction cost: 171827 gas

execution cost: 148723 gas

output: 0x

decoded input: {"string _roomname": "Suite B", "string _roomaddress": "Delhi", "uint256 _rentcost": "3", "uint256 _securitydeposit": "10"}

decoded output: {}

logs: []

raw logs: []

Adding a new room

The screenshot displays a transaction call to the function `Hostel.no_of_rooms()`. The left sidebar shows the transaction details, including the function name `no_of_rooms` and its inputs: `uint256: 1` and `uint256: 2`. The main panel shows the call details, including the sender address `0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2`, the execution cost of 2424 gas, and the output data: `{ "0": "uint256: 2" }`. The logs section shows the raw logs of the transaction.

Num of rooms increased to 2

The screenshot displays a transaction call to the function `Hostel.roomid()`. The left sidebar shows the transaction details, including the function name `roomid` and its inputs: `uint256: roomid 1`, `uint256: agreementid 1`, `string: roomname Suite A`, `string: roomaddress Mumbai`, `uint256: rent_per_month 2`, `uint256: securityDeposit 10`, `uint256: timestamp 1769491254`, `bool: vacant false`, `address: landlord 0x5838D6a701c568545dCfc803Fc8875f56beddC4`, and `address: currentTenant 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2`. The main panel shows the call details, including the sender address `0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2`, the execution cost of 2424 gas, and the output data: `{ "0": "uint256: 1" }`. The logs section shows the raw logs of the transaction.

Calling the room by its room number